Abstract

Resumix is a production-oriented Django and Django REST Framework (DRF) platform that analyzes user resumes against job descriptions and generates actionable improvement feedback. The system supports secure user registration/login with JWT authentication, resume upload in PDF/DOCX format, text extraction, skill-gap analysis, ATS compliance scoring, and ML-based role prediction. The core analysis pipeline computes a final match score using weighted components: keyword similarity, skill match, experience relevance, and ATS compliance. Results are persisted for historical tracking and visualized through a template-based dashboard. Admin users can monitor user activity and system-level statistics through role-protected APIs.

Winter PEP
Project Report

BTECH (CSE – AI and ML)

Submitted To: Sachin Garg

From 21st of January to 28th of February

Submitted By:

Kamal Swarnkar
Registration Number: **12319379**

Introduction

Recruitment workflows are increasingly ATS-driven, and candidates often fail screening due to weak keyword alignment, missing skills, or poor formatting. Resumix is designed to bridge this gap by helping users measure and improve resume quality against specific job requirements. The application combines NLP-driven text processing with supervised machine learning for role prediction.

Summary

Resumix is an resume analysis system built on Django, DRF, PostgreSQL, and NLP/ML tooling. It provides secure authentication, resume parsing, job-description matching, ATS scoring, skill-gap insights, and role prediction. The platform offers transparent, weighted scoring and stores historical analyses for progress tracking. It is suitable for academic submission and can be extended toward enterprise-grade recruitment workflows.

6. Scope and Limitations

Scope:
- JWT-based auth flow
- Resume upload and parsing (.pdf/.docx, max 5MB)
- Job-description-based scoring and suggestions
- Role prediction using trained ML artifacts
- Admin APIs for users and system stats

Limitations:
- Rule-based suggestions (no active LLM personalization)
- English-focused NLP
- Heuristic experience extraction
- Model quality depends on dataset diversity

7. Existing System Overview

Existing tools are often generic or opaque. Resumix improves transparency through weighted scoring, explicit missing-skill output, and modular deployable architecture.

8. Proposed System Architecture and Workflow

Architecture:
- Presentation: Django templates
- API Layer: DRF endpoints
- Business Layer: NLP scoring + ML inference
- Data Layer: PostgreSQL
- Async-ready: Celery + Redis

Workflow:
1. User registers/logs in
2. Uploads resume
3. Text extraction runs
4. User submits job description
5. NLP scores + ML role prediction run
6. Analysis is stored and shown in dashboard/history

9. Module-wise Description

- users: custom user model, login/register, permissions
- resumes: upload, validation, parsing, history
- analysis: scoring, suggestions, persistence, admin stats
- ml: role prediction service and training pipeline
- utils: NLP preprocessing and scoring helpers

10. Technology Stack Justification

Python 3.11+,
Django 5,
DRF,
PostgreSQL,
SimpleJWT,
spaCy,
NLTK,
scikit-learn,
pdfplumber,
python-docx,
drf-spectacular,
Celery, Redis.

11. System Requirements

Software:

Python 3.11+,
PostgreSQL 14+,
optional Redis.

Hardware:

4-core CPU,
8GB RAM minimum,
10GB storage.

12. Database Design

Entities:

- User
- Resume (FK to User)
- Analysis (FK to Resume)

Relationships:

- One User -> Many Resumes
- One Resume -> Many Analyses

13. API/Backend Design Summary

Endpoints:
- POST /api/auth/register/
- POST /api/auth/login/
- POST /api/auth/refresh/
- POST /api/resume/upload/
- POST /api/resume/analyze/
- GET /api/resume/history/
- GET /api/admin/users/
- GET /api/admin/stats/
- GET /api/docs/

14. Frontend Design Summary

Template-based pages: landing, register, login, dashboard, upload, analysis. UI focuses on clarity and analysis readability.

15. AI/ML/NLP Component

NLP computes keyword similarity, skills found/missing, ATS compliance, and experience relevance. ML predicts role from resume text using trained TF-IDF + classifier artifacts.

Final Score = 0.40*Keyword Similarity + 0.30*Skill Match + 0.20*Experience Relevance + 0.10*ATS Compliance

16. Security and Privacy

JWT authentication, role-based admin access, file/input validation, CSRF middleware, and logging for observability.

17. Testing Strategy and Sample Cases

Testing includes unit, API, integration, and regression testing.

Sample cases: valid registration, invalid login, invalid upload format, successful analyze flow, unauthorized admin access.

Problem Statement

Job seekers struggle to evaluate whether their resumes are aligned with a specific job description and ATS expectations. A reliable, secure, and explainable platform is needed to parse resumes, compare with role requirements, identify skill gaps, provide practical suggestions, and track progress over time.

Objective

Primary Objectives:
- Build an end-to-end resume analysis platform
- Provide a quantified resume-job match score
- Offer skill-gap and ATS-focused feedback
- Predict the most likely job role from resume text

Secondary Objectives:
- Maintain user-wise analysis history
- Provide admin analytics for adoption and outcomes
- Expose clear API documentation for integration and scaling

21. Conclusion

Resumix is a complete and practical resume analysis platform combining secure APIs, NLP scoring, ML role prediction, and historical analytics.

22. Future Enhancements

- LLM-based personalized suggestions
- Multilingual resume support
- Recruiter dashboards
- Job portal integration

18. Results/Expected Outcomes

Faster resume evaluation, measurable JD alignment, actionable skill-gap feedback, and trackable profile improvement.

19. Advantages Over Existing Solutions

Transparent scoring, integrated NLP+ML, admin analytics, and modular architecture suitable for academic and production evolution.

20. Timeline/Implementation Plan

Phase 1: Requirements and architecture
Phase 2: Backend and auth
Phase 3: Resume processing and NLP
Phase 4: ML training and inference
Phase 5: UI integration
Phase 6: Testing and hardening
Phase 7: Documentation and final review